

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



BÁO CÁO

**PHÁT TRIỂN HỆ THỐNG GIÁM SÁT VÀ ĐIỀU
KHIỂN NHIỆT ĐỘ CHO NHÀ KÍNH SỬ DỤNG STM32**

Nhóm 13

Giảng viên hướng dẫn: TS. Hoàng Gia Hưng

Hà Nội 2026

Danh sách thành viên và phân công

Họ và tên	Mã số sinh viên	Nhiệm vụ
Trịnh Quang Sáng	23020867	Viết code cho module sensor, software timer.
Phạm Trung Sỹ	23020869	Thiết kế thuật toán PID để điều khiển bộ gia nhiệt.
Nguyễn Tất Thành	23020885	Viết driver cho PWM, GPIO, module actuator.
Vũ Văn Tiến	23020873	Viết module giao tiếp giữa vi điều khiển và máy tính. Thiết kế giao diện cho người dùng trên máy tính.

LỜI CẢM ƠN

Nhóm chúng em xin gửi lời cảm ơn chân thành đến kỹ sư Đỗ Quang Tiến, hiện đang công tác tại FPT Software, đã dành thời gian hỗ trợ, chia sẻ kinh nghiệm thực tế và hướng dẫn chúng em trong quá trình thực hiện dự án. Những góp ý chuyên môn và định hướng của anh đã giúp nhóm hiểu rõ hơn về quy trình phát triển phần mềm nhúng, từ đó hoàn thiện hệ thống một cách hiệu quả và có tính thực tiễn cao hơn.

Sự hỗ trợ nhiệt tình của anh là nguồn động viên lớn giúp chúng em vượt qua những khó khăn trong quá trình nghiên cứu, thiết kế và triển khai dự án. Chúng em xin trân trọng cảm ơn và kính chúc anh nhiều sức khỏe, thành công trong công việc và cuộc sống.

MỤC LỤC

1. MỤC TIÊU BÁO CÁO	1
1.1. Đặt vấn đề	1
1.2. Mục tiêu dự án	1
1.3. Tư duy thiết kế	2
2. TỔNG QUAN HỆ THỐNG	2
3. PHÂN TÍCH YÊU CẦU HỆ THỐNG VÀ CHỌN LINH KIỆN PHÙ HỢP	3
4. KIẾN TRÚC PHẦN MỀM TỔNG THỂ	6
4.1. Tổng quan kiến trúc phần mềm	6
4.2. Thiết kế các module	7
4.2.1. Phân chia module chức năng	7
4.2.2. Tổ chức mã nguồn và cấu trúc thư mục	8
4.3. Nguyên tắc thiết kế phần mềm	10
4.4. Luồng xử lý dữ liệu của hệ thống	10
5. THIẾT KẾ MODULE	11
5.1. Module cảm biến	11
5.1.1. Nguyên lý đo nhiệt độ	12
5.1.2. Định dạng khung dữ liệu	12
5.1.3. Quy trình giao tiếp	12
5.1.4. Triển khai phần mềm	13
5.2. Module điều khiển	14
5.2.1. Triển khai thuật toán PID trên miền thời gian rời rạc	14
5.2.2. Xử lý ngoại lệ và an toàn hệ thống	15
5.3. Module cơ cấu chấp hành	15
5.3.1. Chức năng của Module	15
5.3.2. Cấu trúc dữ liệu	16
5.4. Module giao tiếp	18
5.4.1. Kiến trúc phân tầng module giao tiếp	19
5.4.2. Kỹ thuật tối ưu bộ nhớ và chống nhiễu	20

5.4.3.	Cơ chế máy trạng thái và xử lý ngoại lệ.....	20
5.5.	Định thời và lập lịch	22
5.5.1.	Mục tiêu và kiến trúc	22
5.5.2.	Cấu trúc dữ liệu SoftTimer_t	22
5.5.3.	Các hàm giao tiếp	22
6.	LUỒNG HOẠT ĐỘNG	23
6.1.	Chu trình định thời phi chiếm quyền	23
6.2.	Luồng xử lý dữ liệu	24
6.3.	Chế độ vận hành	26
6.4.	Cơ chế bảo vệ hệ thống.....	28
7.	CÁCH THỨC GIAO TIẾP VỚI PHẦN MỀM	29
7.1.	Định dạng khung truyền	29
7.2.	Kiến trúc và luồng xử lý dữ liệu	29
8.	KIỂM THỬ VÀ ĐÁNH GIÁ.....	32
8.1.	Kiểm thử tầng cảm biến và định thời	32
8.1.1.	Kiểm thử driver DHT22	32
8.1.2.	Kiểm thử module SoftTimer	33
8.2.	Kiểm thử tầng cơ cấu chấp hành.....	34
8.3.	Kiểm thử tích hợp hệ thống và giao tiếp.....	36
10.	TÓM TẮT GIÁ TRỊ VÀ BÀI HỌC KINH NGHIỆM	38
10.1.	Tóm tắt giá trị và bài học kinh nghiệm	38
10.2.	Định hướng cải thiện và mở rộng	39
11.	PHỤ LỤC	39
11.1.	Bảng tra cứu Message ID	39
11.2.	Chi tiết cấu hình các chân của vi điều khiển	40
11.3.	Kết quả hoạt động và mã nguồn dự án	41
11.4.	Danh mục từ viết tắt	41

MỤC LỤC HÌNH ẢNH

Hình 2.1: Sơ đồ tổng quan hệ thống.....	3
Hình 3.1: Vi điều khiển STM32F103C8T6.....	4
Hình 3.2: Module chuyển đổi USB to TTL.....	4
Hình 3.3: Module cảm biến nhiệt độ và độ ẩm (DHT22)	5
Hình 3.4: Motor DC 12V.....	6
Hình 3.5: Module Mosfet	6
Hình 3.6: Điện trở sưởi.....	6
Hình 3.7: Nhiệt kế mini	6
Hình 4.1: Kiến trúc phần mềm tổng thể của hệ thống.....	7
Hình 4.2: Cấu trúc thư mục dự án	8
Hình 4.3: Quan hệ phụ thuộc giữa các tầng	10
Hình 4.4: Các nguyên tắc thiết kế phần mềm.....	10
Hình 4.5: Luồng xử lý dữ liệu của hệ thống.....	11
Hình 5.1: Luồng đọc cảm biến DHT22	13
Hình 5.2: Khối điều khiển PID.....	14
Hình 5.3: Kiến trúc điều khiển cơ cấu chấp hành	16
Hình 5.4: Kiến trúc phân tầng module giao tiếp	20
Hình 5.5: Máy trạng thái nhận Frame	21
Hình 5.6: Luồng logic máy trạng thái xử lý từng byte UART và cơ chế Timeout.....	22
Hình 5.7: Quá trình xử lý của Software Timer	22
Hình 6.1: Luồng hoạt động trong vòng lặp vô hạn.....	24
Hình 6.2: Luồng xử lý dữ liệu	25
Hình 6.3: Luồng hoạt động chế độ tự động.....	26
Hình 6.4: Luồng hoạt động chế độ điều khiển bằng tay.....	27
Hình 6.5: Luồng xử lý lỗi và chế độ an toàn	28
Hình 7.1: Cấu trúc khung truyền giao tiếp	29
Hình 7.2: Sơ đồ luồng giao tiếp bất đồng bộ giữa máy tính và vi điều khiển.....	30

Hình 7.3: Trạng thái hoạt động của giao diện	31
Hình 7.4: Giao diện chính của chương trình trên máy tính.....	31
Hình 8.1: Ảnh chụp phần Frame Log đang hiển thị chuỗi byte TX/RX liên tục	37

MỤC LỤC BẢNG

Bảng 4.1: Mô tả chức năng của các thư mục.....	9
Bảng 7.1: Các thành phần của một khung truyền.....	29
Bảng 8.1: Kiểm thử module DHT22	32
Bảng 8.2: Kiểm thử Software Timer	33
Bảng 8.3: Kiểm thử cơ cấu chấp hành.....	34
Bảng 8.4: Kiểm thử tích hợp hệ thống và giao tiếp.....	36
Bảng 11.1: Bảng tra cứu Message ID	39
Bảng 11.2: Bảng cấu hình các chân của vi điều khiển	40
Bảng 11.3: Bảng danh mục từ viết tắt	41

1. MỤC TIÊU BÁO CÁO

1.1. Đặt vấn đề

Trong các ứng dụng IoT và hệ thống nhúng hiện đại, việc sử dụng các thư viện cấp cao (như HAL, Arduino framework) giúp đẩy nhanh tốc độ phát triển nhưng lại che giấu đi toàn bộ cơ cấu hoạt động bên trong của phần cứng. Điều này dẫn đến việc lập trình viên bị hạn chế khả năng kiểm soát hệ thống ở tầng thanh ghi và gặp nhiều khó khăn khi cần tối ưu hóa hiệu năng, quản lý tài nguyên bộ nhớ hoặc xử lý lỗi chuyên sâu.

Để giải quyết hạn chế trên, dự án này được thực hiện với trọng tâm cốt lõi là thiết kế và kiến trúc phần mềm nhúng trên nền tảng STM32 theo hướng tiếp cận lập trình thanh ghi. Các thiết bị ngoại vi cụ thể (cảm biến DHT22, quạt, thiết bị sưởi) không phải là mục tiêu thiết kế của dự án, mà chỉ đóng vai trò là môi trường vật lý để tương tác và kiểm thử. Việc đo lường thông số điều khiển các thiết bị được thực hiện nhằm mục đích đánh giá tính đúng đắn của logic phần mềm, độ ổn định của các driver tự xây dựng và khả năng đáp ứng của hệ thống.

1.2. Mục tiêu dự án

Dự án được triển khai nhằm đáp ứng ba mục tiêu chức năng chính:

- Thiết kế trình điều khiển thiết bị: Tự xây dựng các thư viện giao tiếp cấp thấp hoàn toàn bằng cách can thiệp trực tiếp và thanh ghi của vi điều khiển thay vì dùng thư viện có sẵn. Cảm biến nhiệt độ được sử dụng như một đối tượng đầu vào để kiểm chứng khả năng thu thập, giải mã và xử lý tín hiệu dữ liệu của các driver này.
- Xây dựng và đóng gói thuật toán điều khiển: Thiết kế, lập trình và chuẩn hóa các module thuật toán xử lý logic thành các khối mã nguồn độc lập, có tính cấu trúc và khả năng tái sử dụng cao. Hệ thống quạt và bộ gia nhiệt bên ngoài đóng vai trò là công cụ trực quan hóa để kiểm thử hiệu suất, độ chính xác và khả năng duy trì trạng thái của các thuật toán đã viết.

- Phát triển kiến trúc giám sát và giao tiếp với máy tính: Xây dựng cơ chế truyền thông hai chiều giữa vi điều khiển và máy tính ở mức thanh ghi. Kênh giao tiếp này hoạt động như một công cụ kiểm thử phần mềm, cho phép theo dõi luồng dữ liệu, đánh giá trạng thái bộ nhớ và tinh chỉnh các tham số logic theo thời gian thực nhằm đảm bảo phần mềm hoạt động ổn định và đúng thiết kế.

1.3. Tư duy thiết kế

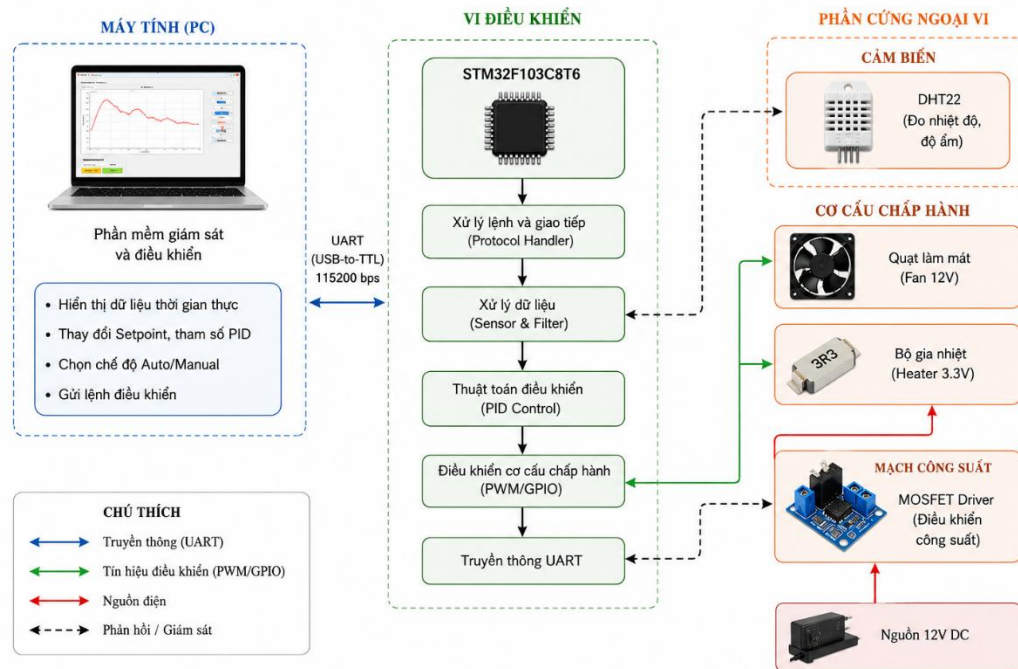
Hệ thống phần mềm được xây dựng dựa trên hai nguyên tắc cốt lõi:

- Phân tách lớp: Phân định rõ ràng giữa tầng phần cứng, tầng trừu tượng phần cứng và tầng ứng dụng. Thay đổi ở tầng phần cứng không làm ảnh hưởng đến logic điều khiển ở tầng trên.
- Phân tách module: Mã nguồn được tổ chức thành các module độc lập, mỗi module chịu trách nhiệm cho một tác vụ duy nhất, tạo điều kiện tối ưu hóa bộ nhớ và tái sử dụng mã nguồn.

2. TỔNG QUAN HỆ THỐNG

Dự án xây dựng hệ thống giám sát và điều khiển nhiệt độ nhà kính dựa trên vi điều khiển STM32F103C8T6. Toàn bộ phần mềm được triển khai từ tầng thanh ghi trở lên mà không sử dụng bất kỳ thư viện đóng gói sẵn nào, nhằm đảm bảo sự hiểu biết sâu về phần cứng và khả năng kiểm soát hoàn toàn hành vi hệ thống.

Hình 2.1: Sơ đồ tổng quan hệ thống



Hình 2.1: Sơ đồ tổng quan hệ thống

Phần cứng hệ thống gồm ba thành phần chính: cảm biến nhiệt độ DHT22 giao tiếp qua giao thức 1-Wire, bộ gia nhiệt (Heater) được điều khiển bằng tín hiệu PWM và quạt làm mát (Fan) hoạt động theo chế độ PWM với toàn bộ công suất theo hai trạng thái bật/tắt. Kết nối với máy tính được thực hiện qua giao tiếp UART tốc độ 115200 bps thông qua các gói tin tự định nghĩa.

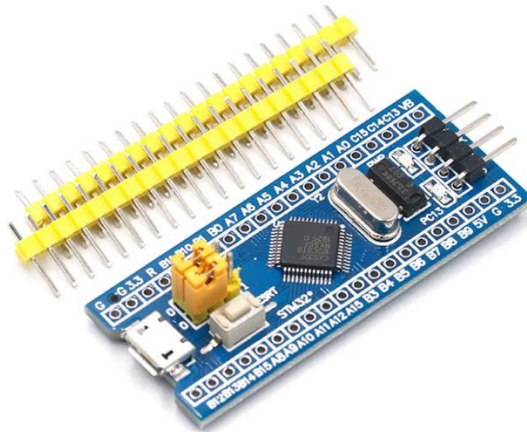
Về mặt phần mềm, hệ thống được tổ chức theo kiến trúc ba tầng - MCU Driver, Platform, Application - kết hợp với các kỹ thuật: thuật toán PID rời rạc với tích phân hình thang, cơ chế lập lịch phi chiếm quyền dựa trên Software Timer và hệ thống hàng đợi gửi và nhận. Chi tiết từng thành phần được trình bày trong các mục tiếp theo.

3. PHÂN TÍCH YÊU CẦU HỆ THỐNG VÀ CHỌN LINH KIỆN PHÙ HỢP

Khối xử lý trung tâm:

Yêu cầu hệ thống: Hệ thống cần một bộ vi điều khiển có năng lực tính toán mạnh mẽ để xử lý các toán tử số thực trong thuật toán PID một cách nhanh chóng, chính xác, giảm trễ hoặc sai lệch chu kỳ lấy mẫu thời gian thực.

Thiết bị phù hợp: Vi điều khiển STM32F103C8T6.



Hình 3.1: Vi điều khiển STM32F103C8T6

Khối giao tiếp với máy tính:

Yêu cầu hệ thống: Thiết lập kênh truyền dữ liệu hai chiều liên tục giữa vi điều khiển và máy tính với tốc độ cao, đảm bảo truyền nhận chính xác các gói tin chứa trạng thái hệ thống và tham số tinh chỉnh PID.

Thiết bị phù hợp: Module chuyển đổi USB to TTL.

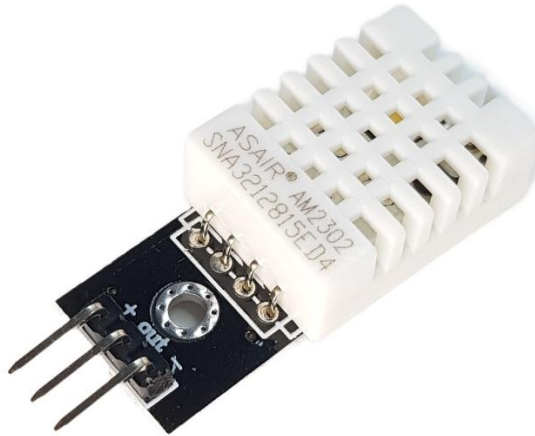


Hình 3.2: Module chuyển đổi USB to TTL

Khối thu thập dữ liệu:

Yêu cầu hệ thống: Đo lường nhiệt độ môi trường liên tục với độ phân giải cao, sai số nhỏ và dải đo rộng.

Thiết bị phù hợp: Module cảm biến nhiệt độ và độ ẩm (DHT22).



Hình 3.3: Module cảm biến nhiệt độ và độ ẩm (DHT22)

Khối cơ cấu chấp hành và đệm công suất:

Yêu cầu hệ thống: Sử dụng các thiết bị gia nhiệt và sinh gió hoạt động an toàn ở điện áp thấp ($\leq 12V$), có khả năng thay đổi công suất tuyến tính bằng xung PWM thông qua một mạch công tắc điện tử đệm dòng cách ly.

Thiết bị phù hợp: Motor DC gắn cánh quạt (12 V), điện trở sưởi 3.3 V , MOSFET.



Hình 3.4: Motor DC 12V



Hình 3.5: Module Mosfet



Hình 3.6: Điện trở sủi

Khối kiểm chứng nhiệt độ:

Yêu cầu hệ thống: Sử dụng nhiệt kế có kích thước nhỏ gọn để dễ dàng tích hợp vào hệ thống và giá cả phải chăng.

Thiết bị phù hợp: Nhiệt kế điện tử mini.



Hình 3.7: Nhiệt kế mini

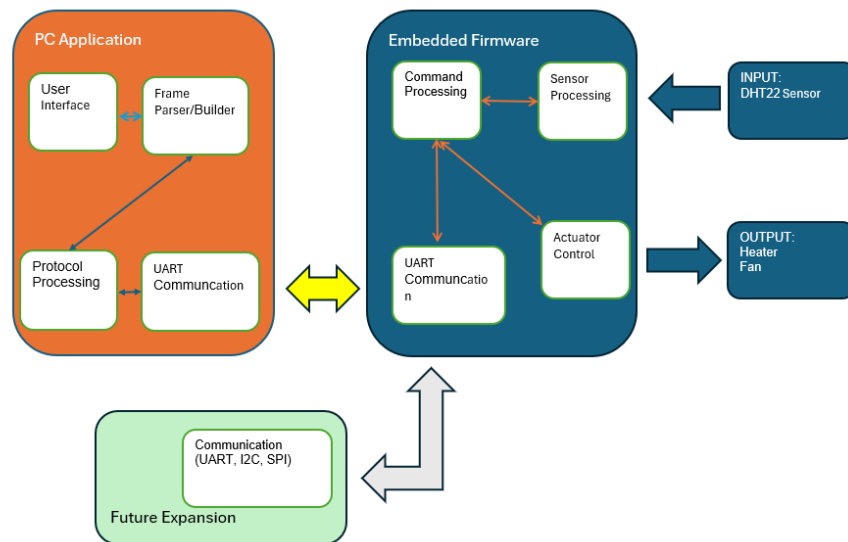
4. KIẾN TRÚC PHẦN MỀM TỔNG THỂ

4.1. Tổng quan kiến trúc phần mềm

Hệ thống được thiết kế theo kiến trúc phân tầng, gồm hai thành phần chính là PC Application và Embedded Firmware, trao đổi dữ liệu với nhau thông qua giao tiếp UART. Chương trình trên máy tính đảm nhiệm chức năng giao diện người dùng, giám sát dữ liệu thời gian thực và gửi các lệnh điều khiển. Firmware trên vi điều

kiến trúc STM32 thực hiện xử lý dữ liệu cảm biến, tính toán thuật toán điều khiển và điều khiển các cơ cấu chấp hành.

Kiến trúc này giúp tách biệt rõ ràng giữa phần mềm giám sát trên máy tính và phần mềm nhúng trên vi điều khiển. Việc phân tách chức năng theo từng tầng giúp hệ thống dễ bảo trì, thuận tiện cho kiểm thử và có khả năng mở rộng thêm các thiết bị hoặc giao thức truyền thông trong tương lai.



Hình 4.1: Kiến trúc phần mềm tổng thể của hệ thống

Trong kiến trúc trên, dữ liệu nhiệt độ được thu thập từ cảm biến DHT22 và đưa vào tầng Application để xử lý. Kết quả điều khiển được chuyển xuống các tầng Platform và MCU Driver để tạo tín hiệu PWM/GPIO điều khiển quạt và bộ gia nhiệt. Đồng thời, trạng thái hoạt động của hệ thống được đóng gói thành các Frame và truyền lên chương trình giám sát trên máy tính.

4.2. Thiết kế các module

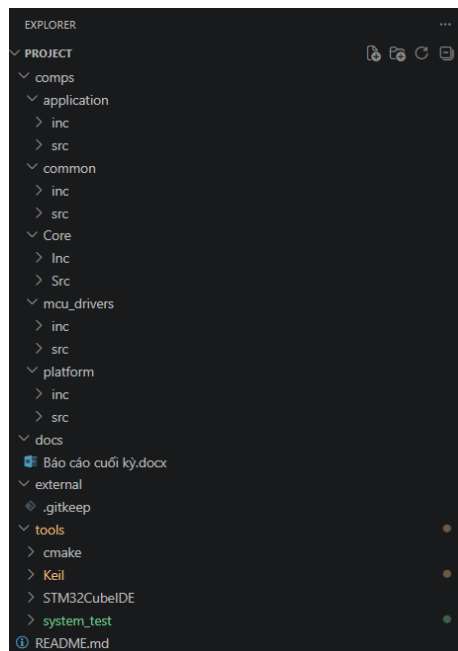
4.2.1. Phân chia module chức năng

Để tăng tính module hóa và khả năng mở rộng, phần mềm được chia thành các module độc lập theo chức năng. Mỗi module chỉ đảm nhận một nhiệm vụ cụ thể và giao tiếp với các module khác thông qua các API đã được chuẩn hóa. Cách tổ chức này giúp giảm sự phụ thuộc giữa các thành phần, thuận tiện cho việc kiểm thử, bảo trì và phát triển thêm các tính năng mới.

Về tổng thể, hệ thống được chia thành hai khối chính gồm PC Application và Embedded Firmware. Trong đó, Firmware tiếp tục được phân thành các module chức năng như đọc cảm biến, điều khiển PID, điều khiển cơ cấu chấp hành, giao tiếp UART và bộ định thời phần mềm.

4.2.2. Tổ chức mã nguồn và cấu trúc thư mục

Để hỗ trợ việc phát triển theo định hướng module, giảm sự phụ thuộc giữa các thành phần và thuận tiện cho việc kiểm thử, mở rộng, toàn bộ mã nguồn của Firmware được tổ chức theo cấu trúc thư mục phân tầng như hình 4.2.

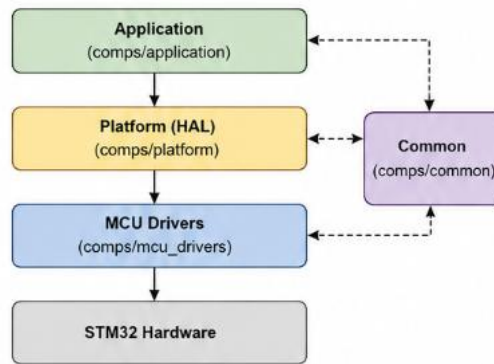


Hình 4.2: Cấu trúc thư mục dự án

Bảng 4.1: Mô tả chức năng của các thư mục

Thư mục	Chức năng
comps/application	Chứa toàn bộ logic nghiệp vụ của hệ thống: xử lý lệnh, điều khiển PID, quản lý các chế độ hoạt động, trạng thái hệ thống...
comps/common	Chứa thành phần dùng chung cho toàn bộ dự án: cấu trúc dữ liệu (Frame_t), kiểu liệt kê, macro, hàng đợi, hàm tiện ích...
comps/platform	Lớp trừu tượng phần cứng do nhóm tự xây dựng: sensor_hal, actuator_hal, comm_hal, soft_timer... cung cấp API cho tầng application sử dụng.
comps/mcu_drivers	Các driver tương tác trực tiếp với thanh ghi của vi điều khiển STM32: GPIO, USART, TIMER (PWM)...
docs	Tài liệu dự án (báo cáo, hướng dẫn sử dụng...)
external	Thư viện bên ngoài (hiện tại trong dự án này chưa có, đây là phần dành cho phát triển trong tương lai).
tools	Các công cụ và file project phục vụ biên dịch, kiểm thử: Keil, STM32CubeIDE, CMake...

Cách tổ chức này giúp ranh giới giữa các tầng được rõ ràng, mỗi module chỉ phụ thuộc vào tầng ngay bên dưới thông qua các API đã được định nghĩa, tránh phụ thuộc vòng và thuận lợi cho việc duy trì và mở rộng hệ thống.



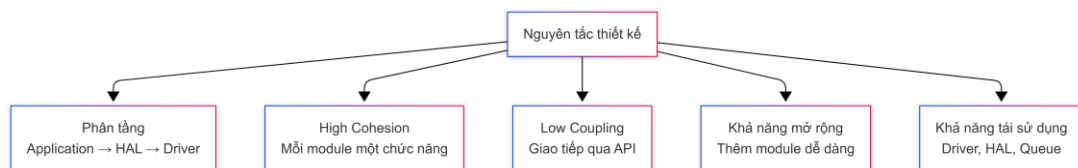
Hình 4.3: Quan hệ phụ thuộc giữa các tầng

4.3. Nguyên tắc thiết kế phần mềm

Trong quá trình phát triển, hệ thống được thiết kế theo hướng module nhằm tăng khả năng mở rộng và thuận tiện cho việc bảo trì. Các thành phần trong hệ thống được phân tầng rõ ràng và chỉ giao tiếp với nhau thông qua các API đã được chuẩn hóa.

Tầng application chỉ chứa thuật toán và logic điều khiển, trong khi các thao tác với phần cứng được thực hiện thông qua tầng platform và mcu_drivers. Cách phân tầng này giúp giảm sự phụ thuộc giữa thuật toán và phần cứng, đồng thời tạo điều kiện thay thế hoặc mở rộng các ngoại vi mà không ảnh hưởng đến tầng ứng dụng.

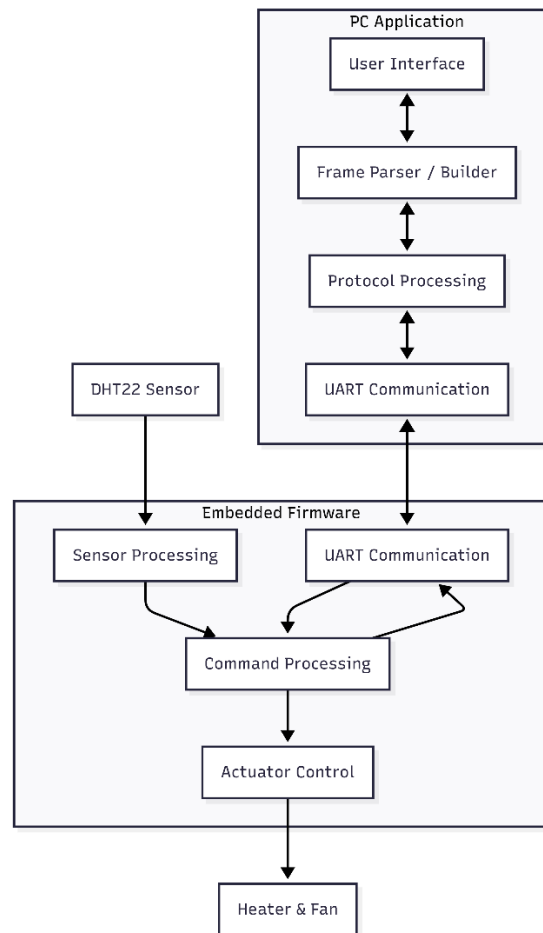
Bên cạnh đó, mỗi module chỉ đảm nhiệm một chức năng riêng biệt và hạn chế phụ thuộc vào các module khác. Nhờ đó, hệ thống có cấu trúc rõ ràng, dễ kiểm thử, dễ bảo trì và thuận tiện cho việc phát triển thêm các chức năng trong tương lai.



Hình 4.4: Các nguyên tắc thiết kế phần mềm

4.4. Luồng xử lý dữ liệu của hệ thống

Dữ liệu nhiệt độ được thu thập từ cảm biến DHT22 và chuyển đến Firmware để xử lý. Sau khi tính toán, kết quả điều khiển được gửi đến quạt và bộ gia nhiệt, đồng thời trạng thái hoạt động được đóng gói thành frame và truyền lên chương trình trên máy tính để hiển thị. Ngược lại, các lệnh từ người dùng được gửi từ giao diện trên máy tính xuống Firmware thông qua UART để cập nhật chế độ hoạt động hoặc thay đổi các tham số điều khiển.



Hình 4.5: Luồng xử lý dữ liệu của hệ thống

5. THIẾT KẾ MODULE

5.1. Module cảm biến

Dự án sử dụng module DHT22 làm cảm biến đo nhiệt độ môi trường, cung cấp dải đo trong khoảng từ -40 độ C đến 80 độ C và độ chính xác $\pm 0,5^{\circ}\text{C}$. Module gồm

3 chân: 1 chân cấp nguồn VCC 3.3V, một chân GND và 1 chân dùng để giao tiếp với vi điều khiển.

5.1.1. Nguyên lý đo nhiệt độ

Phần tử đo nhiệt độ trong DHT22 là Thermistor NTC, đây là một điện trở nhạy nhiệt. Giá trị của nó giảm khi nhiệt độ môi trường tăng. Chip xử lý tích hợp bên trong cảm biến đo giá trị điện trở này. Sau đó, nó sử dụng một thuật toán nội bộ để chuyển thành giá trị nhiệt độ theo thang Celsius. Nguyên lý này đảm bảo độ chính xác cao trong phạm vi hoạt động của cảm biến.

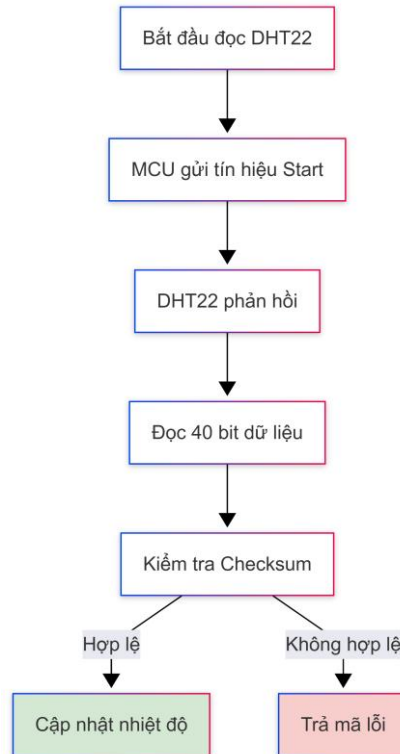
5.1.2. Định dạng khung dữ liệu

Dữ liệu được truyền từ DHT22 tới vi điều khiển dưới dạng một khung 40 bit. Khung dữ liệu này bao gồm 5-byte thông tin. Hai byte đầu tiên chứa dữ liệu độ ẩm, hai byte tiếp theo chứa dữ liệu nhiệt độ và byte cuối cùng là byte kiểm tra tổng (checksum). Byte checksum được sử dụng để xác minh tính toàn vẹn của dữ liệu, là tổng của 4-byte trước đó. Cơ chế kiểm tra này giúp phát hiện lỗi truyền nhận trước khi dữ liệu được đưa vào xử lý.

5.1.3. Quy trình giao tiếp

Quá trình giao tiếp bắt đầu bằng tín hiệu khởi động từ vi điều khiển: kéo đường dữ liệu xuống mức thấp trong ít nhất 1ms, sau đó kéo lên cao và chờ 20–40μs. DHT22 phản hồi bằng cách kéo đường dữ liệu xuống thấp 80μs rồi kéo lên cao 80μs, báo hiệu sẵn sàng truyền dữ liệu.

Sau tín hiệu phản hồi, cảm biến truyền 40-bit theo quy tắc mã hóa thời gian: mỗi bit bắt đầu bằng 50μs ở mức thấp, tiếp theo là mức cao kéo dài 26–28μs nếu là bit 0, hoặc 70μs nếu là bit 1.



Hình 5.1: Luồng đọc cảm biến DHT22

5.1.4. Triển khai phần mềm

Tất cả các hàm delay trong phần sensor_hal đều gây ra hiện tượng block hệ thống, làm lãng phí một phần thời gian sử dụng CPU. Tuy nhiên, điều này là cần thiết do quá trình truyền nhận giữa vi điều khiển và DHT22 diễn ra trong thời gian ngắn, nhằm đảm bảo được tính chính xác và độ tin cậy của dữ liệu.

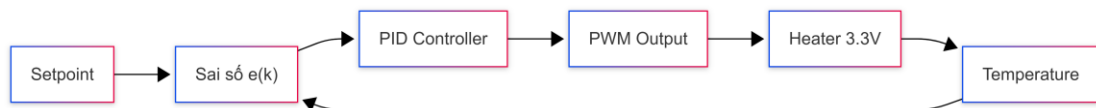
Các hàm chính trong module:

- DHT22_Start(): Gửi tín hiệu khởi động bằng cách cấu hình chân dữ liệu sang chế độ Output, giữ mức thấp 2ms, kéo lên cao và chờ 30 μ s, sau đó chuyển sang chế độ Input để nhận phản hồi.
- DHT22_CheckResponse(): Kiểm tra phản hồi từ cảm biến tại thời điểm 120 μ s sau tín hiệu khởi động. Trả về 1 nếu cảm biến phản hồi đúng, 0 nếu không có tín hiệu.

- DHT22_Read_Byte(): Đọc 8 bit dữ liệu liên tiếp bằng cách kiểm tra mức tín hiệu tại thời điểm 40μs sau khi mỗi bit được kéo lên cao, xác định giá trị bit 0 hoặc 1 tương ứng.
- DHT22_ReadData(): Điều phối toàn bộ quy trình đọc bằng cách gọi tuần tự các hàm trên, đọc đủ 5 byte, kiểm tra checksum, rồi chuyển đổi dữ liệu nhị phân thành giá trị số thực và cập nhật vào bộ nhớ.
- ReadTemperature(): Trả về giá trị nhiệt độ hiện tại đã được cập nhật bởi DHT22_ReadData().

5.2. Module điều khiển

5.2.1. Triển khai thuật toán PID trên miền thời gian rời rạc



Hình 5.2: Khối điều khiển PID

Cơ chế: Tác động ngược – nhiệt độ giảm thì công suất sưởi tăng. Chu kỳ lấy mẫu $T = 100 \text{ ms}$

Sai lệch hệ thống:

$$e(k) = \text{Setpoint} - y(k)$$

Tín hiệu ngõ ra:

$$u(k) = P(k) + I(k) + D(k)$$

Khâu tỷ lệ (P) :

$$P(k) = K_p e(k)$$

Khâu tích phân hình thang (I): Giảm tích lũy sai số cắt của phương pháp Euler tiến:

$$I(k) = I(k-1) + \frac{1}{2} K_i T [e(k) + e(k-1)]$$

Khâu vi phân trên tín hiệu đo và lọc thông thấp (D): Triệt tiêu hiện tượng dội xung khi đổi Setpoint đột ngột và lọc nhiễu cao tần qua hằng số thời gian τ :

$$D(k) = 2Kd [y(k) - y(k-1)] + (2\tau - T)D(k-1) / 2\tau + T$$

(Tham số mặc định nhóm tinh chỉnh: $Kp = 2,5 \cdot Ki = 0,1 \cdot Kd = 0,4 \cdot T = 0,1s \cdot \tau = 0,05s$.)

5.2.2. Xử lý ngoại lệ và an toàn hệ thống

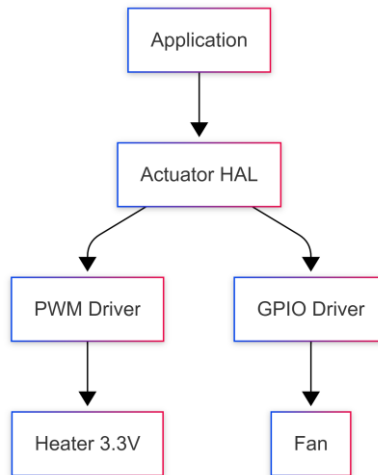
Hai cơ chế bảo vệ được tích hợp trong module điều khiển:

- Giới hạn tích phân: Bộ nhớ đệm tích phân $I(k)$ được giới hạn trong dải $[-40,0; 40,0]$ để ngăn hiện tượng tích lũy sai số kéo dài khi hệ thống chưa đạt giá trị đặt trong thời gian dài.
- Khóa an toàn sinh học: Khi nhiệt độ đo được nằm ngoài dải hoạt động của cảm biến (dưới -40°C hoặc trên 80°C), hệ thống nhận định đường truyền cảm biến có sự cố, lập tức đặt công suất gia nhiệt về 0 và khóa trạng thái thiết bị, đồng thời gửi mã lỗi lên máy tính để cảnh báo.

5.3. Module cơ cấu chấp hành

5.3.1. Chức năng của Module

Actuator Module là thành phần chịu trách nhiệm điều khiển các thiết bị đầu ra của hệ thống. Sau khi tầng Application tính toán được giá trị điều khiển, dữ liệu sẽ được chuyển xuống Actuator Module để tạo ra tín hiệu phân cứng tương ứng.



Hình 5.3: Kiến trúc điều khiển cơ cấu chấp hành

Trong phạm vi dự án, module hỗ trợ hai cơ cấu chấp hành chính:

- Quạt làm mát (Fan).
- Bộ gia nhiệt (Heater).

Đối với Heater, hệ thống sử dụng tín hiệu PWM để điều chỉnh công suất hoạt động. Duty Cycle của tín hiệu PWM càng lớn thì công suất gia nhiệt càng cao.

Đối với Fan, hệ thống cho phép điều khiển thông qua tín hiệu PWM hoặc chế độ bật/tắt, tùy theo yêu cầu điều khiển từ tầng Application.

Nhờ việc tách riêng Actuator Module, các phần còn lại của hệ thống không cần quan tâm tới chi tiết phần cứng như Timer, GPIO hay thanh ghi điều khiển của vi điều khiển STM32.

5.3.2. Cấu trúc dữ liệu

a. Cấu trúc dữ liệu kiểu `ActuatorDevice_t`

Mục đích: Xác định thiết bị cần điều khiển.

Các giá trị được định nghĩa:

- `ACTUATOR_FAN`: Quạt làm mát.
- `ACTUATOR_HEATER`: Bộ gia nhiệt.

- ACTUATOR_MICRO: Thiết bị mở rộng.

- ACTUATOR_LIGHT: Thiết bị mở rộng.

Trong phiên bản hiện tại của hệ thống, hai thiết bị được sử dụng là Fan và Heater.
Việc sử dụng kiểu liệt kê dễ dàng mở rộng thêm các thiết bị mới trong tương lai.

b. Cấu trúc dữ liệu kiểu ActuatorMode_t

Mục đích: Xác định phương thức điều khiển của thiết bị.

Các chế độ hỗ trợ:

- ACT_MODE_PWM: Điều khiển bằng tín hiệu PWM.

- ACT_MODE_ONOFF: Điều khiển bật hoặc tắt thiết bị.

Chế độ PWM thường được sử dụng khi cần thay đổi công suất hoạt động liên tục.

Chế độ ON/OFF phù hợp với các thiết bị chỉ cần hai trạng thái hoạt động.

c. Cấu trúc dữ liệu kiểu ActuatorUnit_t

Mục đích: Xác định đơn vị của giá trị điều khiển được gửi từ tầng Application.

Đối với Fan:

- UNIT_PERCENT: Giá trị theo phần trăm công suất (0–100%).

- UNIT_RPM: Giá trị theo số vòng quay trên phút.

- UNIT_RPS: Giá trị theo số vòng quay trên giây.

Đối với Heater:

- UNIT_PERCENT: Giá trị theo phần trăm công suất.

- UNIT_DEG_C: Giá trị nhiệt độ theo độ C.

- UNIT_DEG_F: Giá trị nhiệt độ theo độ F.

- UNIT_DEG_K: Giá trị nhiệt độ theo độ K.

Các đơn vị này sẽ được chuyển đổi về phần trăm công suất trước khi tạo tín hiệu PWM.

Cách thiết kế này giúp tăng Application có thể làm việc với các đơn vị gần với thực tế thay vì phải tự tính toán Duty Cycle.

d. Cấu trúc dữ liệu kiểu `ActuatorPrms_struct_t`

Mục đích: Đóng gói toàn bộ thông tin cần thiết cho một yêu cầu điều khiển.

Các thành phần:

- `index`: Xác định thiết bị cần điều khiển.
- `mode`: Xác định chế độ điều khiển.
- `unit`: Xác định đơn vị dữ liệu.
- `value`: Giá trị điều khiển.

Đây là cấu trúc dữ liệu được truyền trực tiếp vào hàm `ActuatorHAL_Set()`.

Việc sử dụng một cấu trúc duy nhất giúp giảm số lượng tham số truyền vào hàm, đồng thời dễ dàng mở rộng thêm các thuộc tính mới trong tương lai.

e. Cấu trúc dữ liệu kiểu `errorCode`

Mục đích: Chuẩn hóa kết quả trả về của module Actuator HAL.

Các mã trạng thái:

- `ACTUATOR_OK`: Thực hiện thành công.
- `ACTUATOR_ERROR_INDEX`: Thiết bị điều khiển không hợp lệ.
- `ACTUATOR_ERROR_MODE`: Chế độ điều khiển không hợp lệ.
- `ACTUATOR_ERROR_UNIT`: Đơn vị dữ liệu không phù hợp với thiết bị.
- `ACTUATOR_ERROR_VALUE`: Giá trị điều khiển vượt giới hạn cho phép.

Khi phát hiện lỗi, module sẽ đưa các đầu ra PWM về trạng thái an toàn trước khi trả mã lỗi cho tăng Application.

Cơ chế này giúp tăng độ tin cậy và hạn chế các tác động không mong muốn lên phần cứng.

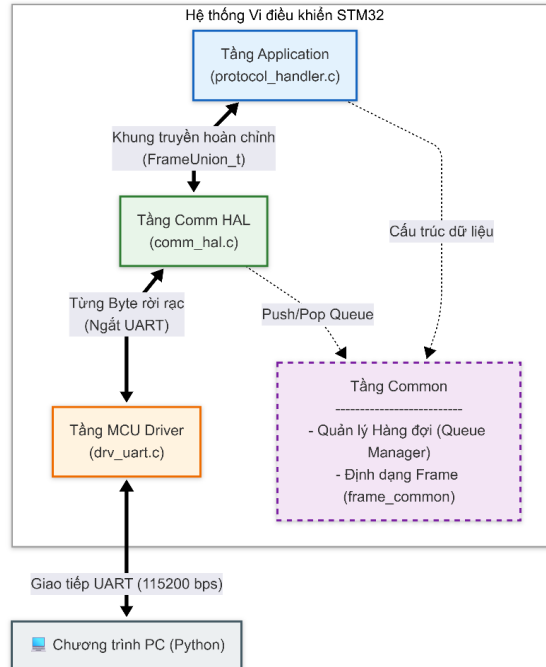
5.4. Module giao tiếp

Module giao tiếp đảm nhận việc truyền nhận dữ liệu hai chiều giữa vi điều khiển và máy tính. Đảm bảo khả năng mở rộng và không gây nghẽn CPU, module được thiết kế phân tầng và áp dụng các kỹ thuật quản lý bộ nhớ linh hoạt.

5.4.1. Kiến trúc phân tầng module giao tiếp

Để đảm bảo khả năng mở rộng, tránh lỗi phụ thuộc vòng và nghẽn CPU, module giao tiếp được thiết kế phân tách thành 4 lớp chuyên biệt. Đặc biệt, hệ thống thiết kế một tầng riêng biệt cho các tài nguyên dùng chung.

- Tầng MCU Driver (drv_uart): Giao tiếp trực tiếp với thanh ghi phần cứng. Khởi tạo USART1 (với tốc độ baud được định nghĩa ở tầng Comm HAL – tầng khởi tạo MCU Driver) và chứa trình phục vụ ngắt USART1_IRQHandler().
- Tầng Comm HAL (comm_hal): Vận hành máy trạng thái để bắt từng byte rồi rạc ghép thành Frame. Quản lý thao tác với hàng đợi để lấy dữ liệu gửi đi hoặc đẩy dữ liệu vào.
- Tầng Application(protocol_handler): Điều phối logic, phân loại và xử lý gói tin theo GID/MID và liên kết với các module điều khiển khác.
- Tầng Common: Tầng này không trực tiếp điều khiển phần cứng mà cung cấp các tài nguyên toàn cục như: cấu trúc dữ liệu (Frame_t), định nghĩa các trường sử dụng trong gói tin và quản lý hàng đợi vòng (queue_manager.c).



Hình 5.4: Kiến trúc phân tầng module giao tiếp

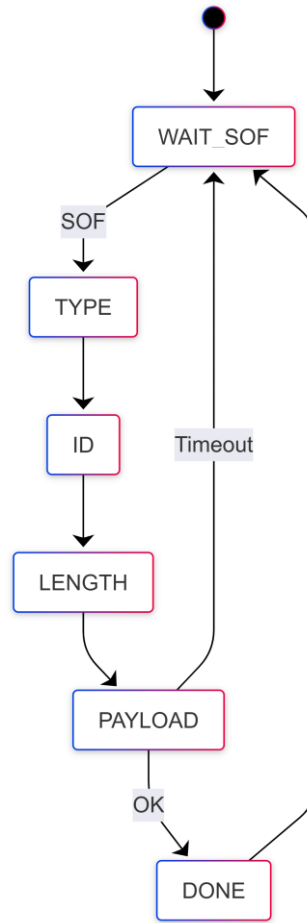
5.4.2. Kỹ thuật tối ưu bộ nhớ và chống nhiễu

Dữ liệu byte thô nhận từ UART được ánh xạ trực tiếp vào cấu trúc `Frame_t` thông qua `FrameUnion_t` (ép kiểu nhanh bằng `Union`) và thuộc tính `__attribute__((packed))`. Điều này giúp việc giải mã diễn ra tức thời ($O(1)$) mà không cần dùng lệnh `memcpy` hay dịch bit thủ công.

Để giải quyết bài toán rớt gói tin khi CPU bận tính toán PID, dữ liệu UART được ngắt phần cứng đẩy thẳng vào `rx_queue`. Vòng lặp chính sẽ lấy `Frame` ra xử lý khi rảnh rỗi.

5.4.3. Cơ chế máy trạng thái và xử lý ngoại lệ

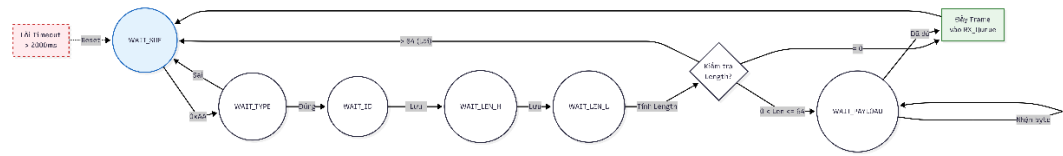
Việc nhận dữ liệu qua UART diễn ra theo từng byte rời rạc. Nếu sử dụng các hàm chờ vòng lặp để đợi đủ khung truyền, hệ thống sẽ bị treo. Thay vào đó, module `Comm HAL` sử dụng máy trạng thái hữu hạn (FSM) bên trong hàm Callback của ngắt UART.



Hình 5.5: Máy trạng thái nhận Frame

Luồng hoạt động: Máy trạng thái mặc định ở WAIT_SOF. Mỗi khi ngắt UART bắt được 1 byte, trạng thái sẽ nhảy bậc dần xuống dưới để thu thập TYPE, ID, LENGTH và PAYLOAD. Chỉ khi thu thập đủ và đúng định dạng, Frame mới được đẩy vào hàng đợi để xử lý. Nếu phát hiện byte lỗi, FSM lập tức hủy Frame hiện tại và quay về WAIT_SOF.

Xử lý ngoại lệ đứt gãy đường truyền: Nếu cáp truyền bị lỏng hoặc PC ngừng gửi dữ liệu giữa chừng, FSM có thể bị kẹt vĩnh viễn ở trạng thái chờ Payload. Để chống lại lỗi này, một bộ định thời mềm (SoftTimer_t s_rx_byte_timeout) được kích hoạt. Nếu sau 2000ms không nhận được byte tiếp theo, Timer sẽ tự động Reset FSM về trạng thái ban đầu, giải phóng bộ nhớ để sẵn sàng nhận lệnh mới.



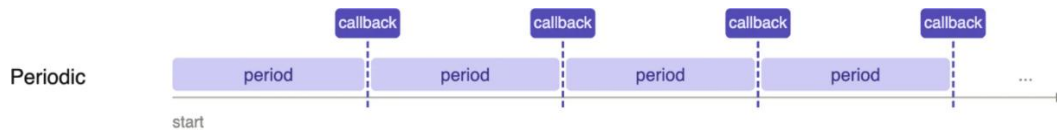
Hình 5.6: Luồng logic máy trạng thái xử lý từng byte UART và cơ chế Timeout

5.5. Định thời và lập lịch

5.5.1. Mục tiêu và kiến trúc

Trong các hệ thống nhúng, việc dùng hàm delay() làm chặn (block) toàn bộ hệ thống, gây lãng phí thời gian sử dụng CPU và hiệu suất hệ thống. Để khắc phục hạn chế này, dự án sử dụng cơ chế Software Timer.

Cơ chế này giúp thực hiện các thao tác theo chu kỳ hoặc sau một khoảng thời gian định trước mà không chiếm dụng thời gian CPU để chờ. Khi một software timer hết thời gian, một hàm gọi lại (callback function) sẽ được thực thi.



Hình 5.7: Quá trình xử lý của Software Timer

5.5.2. Cấu trúc dữ liệu SoftTimer_t

Ta định nghĩa một cấu trúc dữ liệu SoftTimer t gồm các thành phần sau:

- counter: bộ đếm lùi thời gian, khi đếm về 0, hàm callback sẽ được gọi để chạy định kỳ
- period: chu kỳ hoạt động mặc định của tác vụ
- event: cờ đánh dấu, đếm số lần sự kiện xảy ra nhưng chưa được xử lý
- enable: trạng thái bật hoặc vô hiệu hóa software timer
- callback: con trỏ hàm (TimerCallback_t) trỏ tới tác vụ cần thực thi khi bộ đếm hết thời gian

5.5.3. Các hàm giao tiếp

- SoftTimer_Start: khởi tạo, thiết lập software timer với chu kỳ và hàm được thực thi định kỳ
- SoftTimer_Stop: vô hiệu hóa software timer, khi này, hàm callback sẽ không được gọi khi bộ đếm hoàn thành một chu kỳ
- SoftTimer_Update: Hàm này được gọi trong hàm xử lý ngắt của SysTick là SysTick_Handler với tần số cấu hình là 1kHz (mỗi 1ms), đây sẽ là một kênh tham chiếu chuẩn dùng chung cho nhiều software timer. Cứ sau 1 ms, biến count sẽ giảm đi 1 đơn vị, cho đến khi trở về 0 chính là đến thời điểm mà hàm xử lý cần được thực thi định kỳ với chu kỳ là period. Và khi counter đếm về 0, nó tự động nạp lại giá trị period và tăng event lên 1, báo hiệu đã đến lúc chạy tác vụ cần được thực thi
- SoftTimer_Dispatch: Hàm này được sử dụng để tách biệt việc đếm thời gian và việc thực thi code, từ đó tăng độ tin cậy của hệ thống, tránh việc thực thi tác vụ làm delay software timer. Nếu event lớn hơn 0, nó sẽ gọi hàm callback() tương ứng và giảm event đi 1.

Với thiết kế này, tầng Ứng dụng (Application) hoàn toàn không can thiệp và các thanh ghi phần cứng. Từ đó giúp loại bỏ giới hạn về số lượng hữu hạn các Timer phần cứng, tăng khả năng mở rộng và sử dụng linh hoạt.

6. LUỒNG HOẠT ĐỘNG

6.1. Chu trình định thời phi chiếm quyền

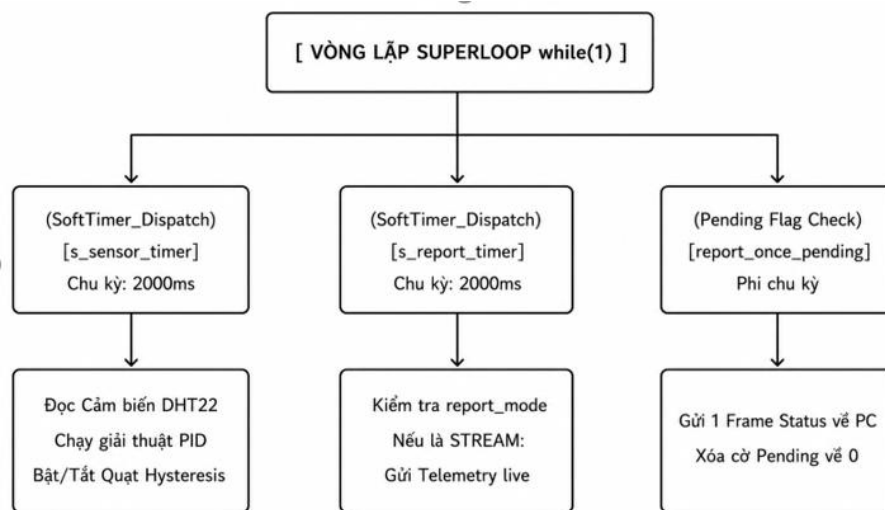
Cơ chế: Tổ chức theo mô hình Superloop kết hợp Bộ điều phối sự kiện (Event Dispatcher). Định thời nền bằng ngắt cứng SysTick chu kỳ 1ms.

Nguyên lý: Sử dụng cờ hiệu (flag) thay thế hoàn toàn cho hàm delay().

Phân phối tác vụ:

- Luồng Cảm biến (2000ms): Kích hoạt cờ s_sensor_due, đọc dữ liệu từ DHT22.

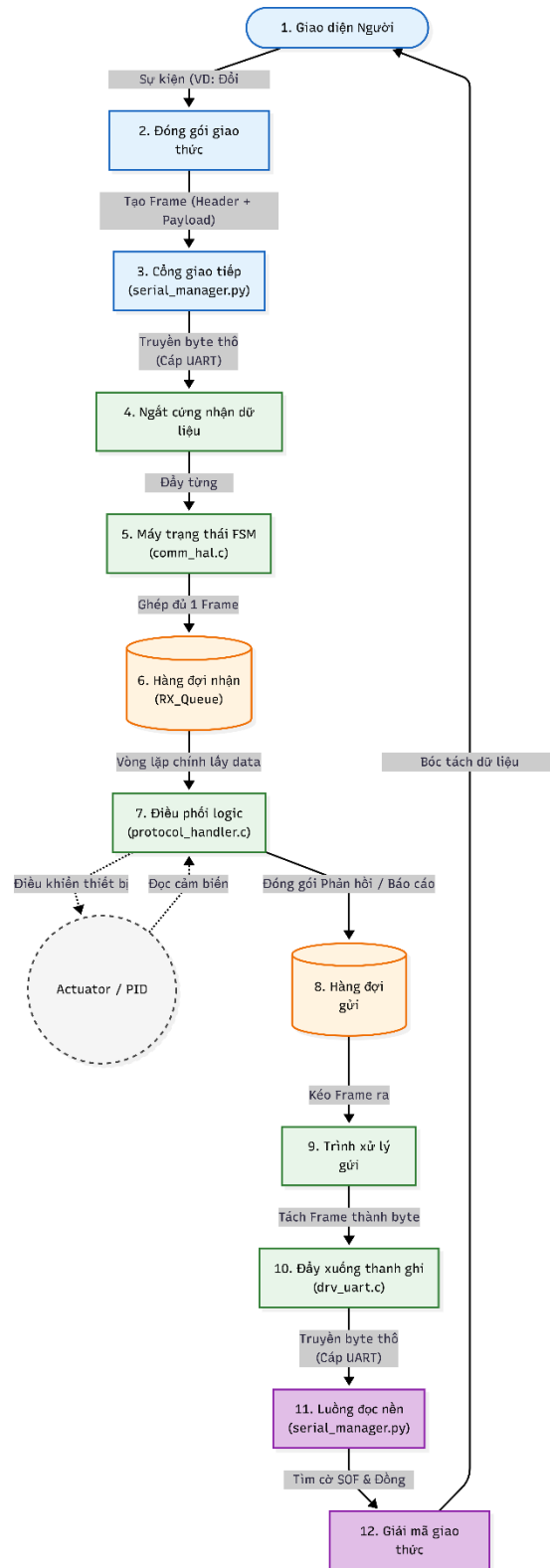
- Luồng Điều khiển (100ms): Kích hoạt cờ s_control_due, tính toán bộ điều khiển PID.
- Luồng Truyền thông (2000ms / Tức thời): Đẩy dữ liệu giám sát (Telemetry) lên PC.



Hình 6.1: Luồng hoạt động trong vòng lặp vô hạn

6.2. Luồng xử lý dữ liệu

Giải pháp: Tách biệt luồng nhận dữ liệu và luồng xử lý thông qua cấu trúc dữ liệu hàng đợi để giảm tỉ lệ mất gói tin.

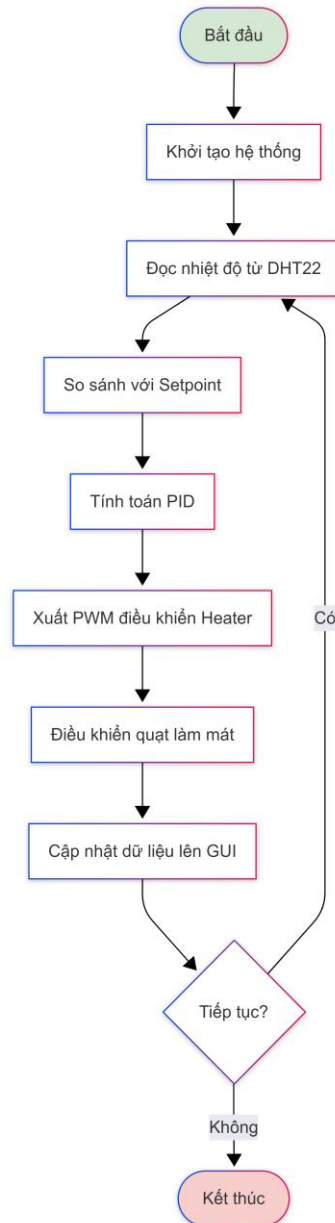


Hình 6.2: Luồng xử lý dữ liệu

6.3. Chế độ vận hành

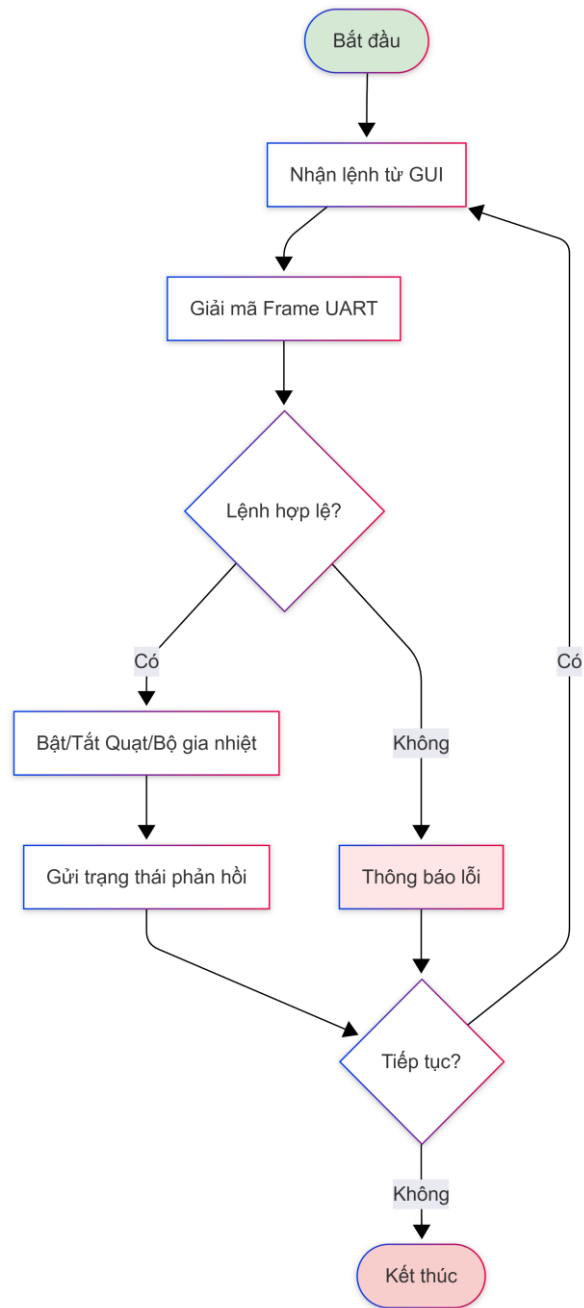
Hệ thống hỗ trợ hai chế độ hoạt động là Auto và Manual.

Ở chế độ Auto, bộ điều khiển PID tự động điều chỉnh công suất gia nhiệt để duy trì nhiệt độ theo giá trị đặt. Quạt làm mát được điều khiển dựa trên ngưỡng nhiệt độ nhằm đảm bảo hệ thống hoạt động ổn định.



Hình 6.3: Luồng hoạt động chế độ tự động

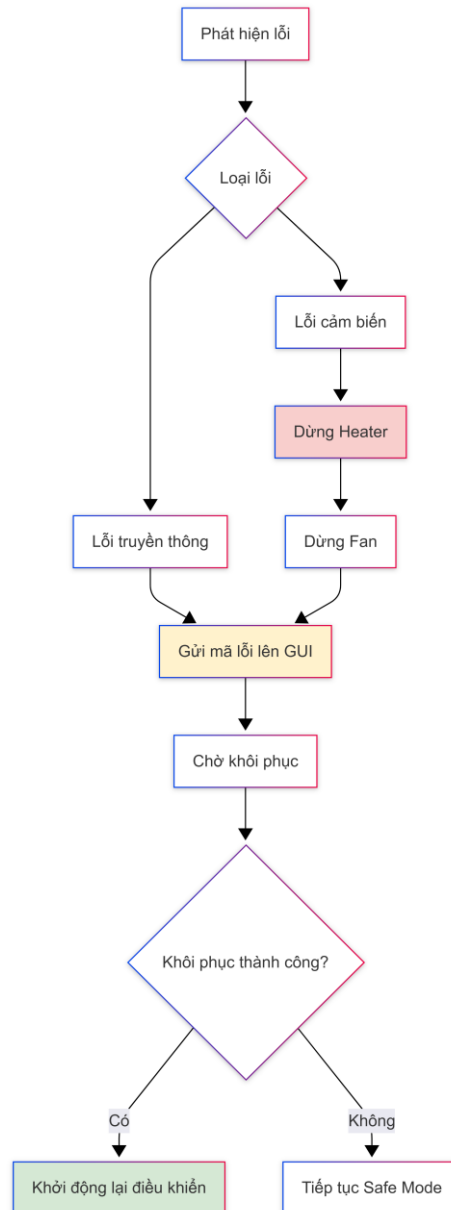
Ở chế độ Manual, người dùng có thể điều khiển trực tiếp quạt và bộ gia nhiệt từ chương trình PC thông qua giao thức truyền thông UART. Trong chế độ này, các tín hiệu điều khiển được gửi trực tiếp đến cơ cấu chấp hành mà không thông qua thuật toán PID.



Hình 6.4: Luồng hoạt động chế độ điều khiển bằng tay

6.4. Cơ chế bảo vệ hệ thống

Để đảm bảo an toàn trong quá trình vận hành, hệ thống liên tục giám sát giá trị nhiệt độ đo được. Khi phát hiện giá trị nằm ngoài phạm vi hoạt động cho phép, hệ thống sẽ ngắt toàn bộ cơ cấu chấp hành và chuyển sang trạng thái an toàn. Đồng thời, thông tin lỗi được gửi đến chương trình PC để thông báo cho người vận hành.



Hình 6.5: Luồng xử lý lỗi và chế độ an toàn

7. CÁCH THỨC GIAO TIẾP VỚI PHẦN MỀM

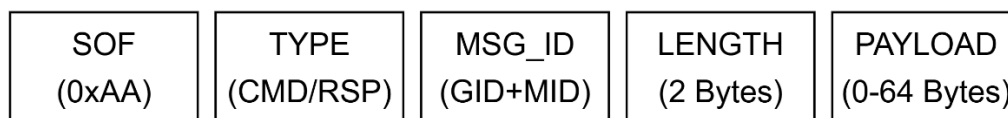
Chương trình giám sát trên máy tính được xây dựng bằng Python/PyQt5, hoạt động như một trạm điều khiển trung tâm. Phần mềm cho phép giám sát thông số thời gian thực và gửi lệnh điều khiển (Setpoint, PID, Auto/Manual...) xuống Firmware.

7.1. Định dạng khung truyền

Để hai hệ thống hiểu nhau, dữ liệu được đóng gói theo một giao thức chuẩn gồm 5-byte Header và tối đa 64-byte Payload.

Bảng 7.1: Các thành phần của một khung truyền

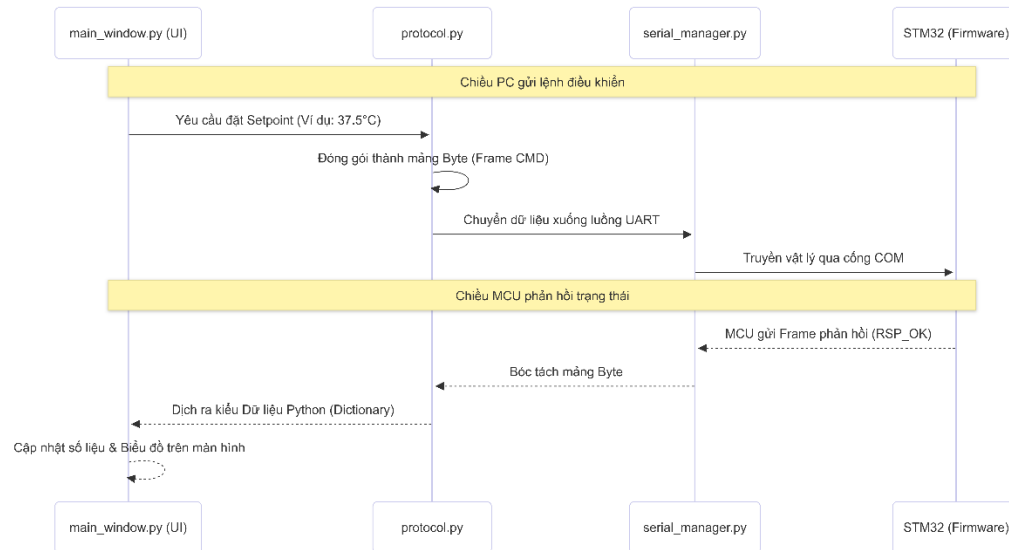
Trường dữ liệu	Kích thước (Byte)	Mô tả chi tiết
SOF	1	Cờ bắt đầu khung truyền. Giá trị cố định: 0xAA.
TYPE	1	Loại bản tin: 0x0F (Lệnh từ PC xuống MCU) hoặc 0x1F (Phản hồi từ MCU lên PC).
ID	1	Mã định danh bản tin. Cấu trúc: 4-bit cao là GID , 4-bit thấp là MID .
LENGTH	2	Chiều dài của phần Payload. Gồm 1-byte cao (LEN_H) và 1-byte thấp (LEN_L).
PAYLOAD	0 - 64	Dữ liệu mang theo (thông số cấu hình, giá trị cảm biến, trạng thái).



Hình 7.1: Cấu trúc khung truyền giao tiếp

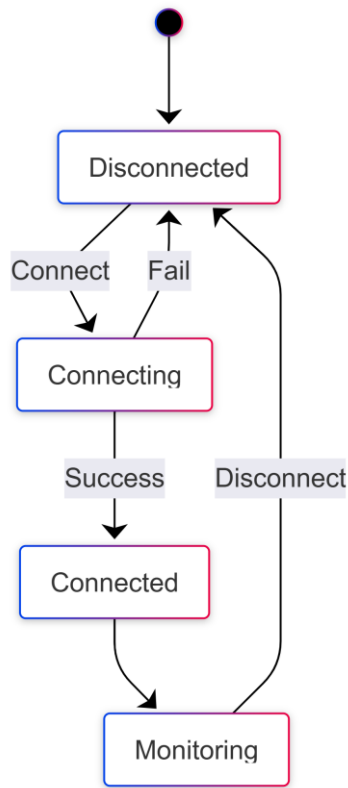
7.2. Kiến trúc và luồng xử lý dữ liệu

Phần mềm trên máy tính được thiết kế phân tách rõ ràng giữa giao diện và giao tiếp phần cứng, giúp luồng giao diện không bị treo khi chờ dữ liệu UART.

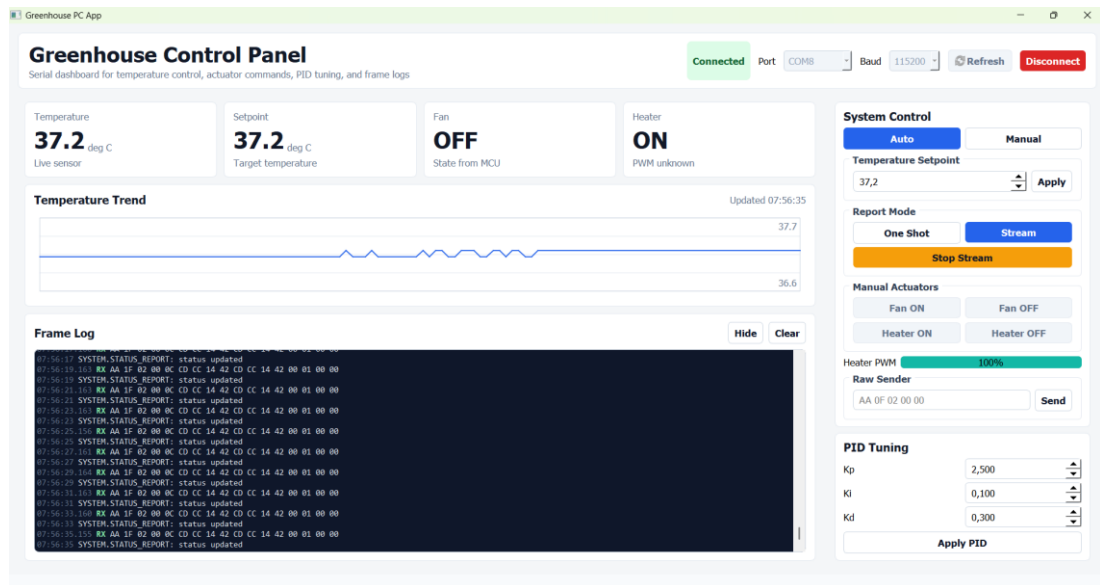


Hình 7.2: Sơ đồ luồng giao tiếp bất đồng bộ giữa máy tính và vi điều khiển

- `serial_manager.py`: Quản lý cổng COM. Chạy ngầm một luồng riêng biệt sử dụng `SerialWorker` để liên tục quét bộ đệm UART, tìm cờ SOF và đồng bộ hóa luồng dữ liệu.
- `protocol.py`: Bộ dịch mã. Cung cấp các hàm `build_frame()` để đóng gói lệnh và `parse_frame()` để giải mã dữ liệu nhị phân thành các tham số cấu hình.
- `main_window.py`: Giao diện đồ họa. Cập nhật biểu đồ biến thiên nhiệt độ bằng `QPainter` và hiển thị các gói tin đã truyền/nhận ở Frame Log.



Hình 7.3: Trạng thái hoạt động của giao diện



Hình 7.4: Giao diện chính của chương trình trên máy tính

8. KIỂM THỬ VÀ ĐÁNH GIÁ

8.1. Kiểm thử tăng cảm biến và định thời

Mục tiêu của phần kiểm thử này là xác minh rằng driver giao tiếp với cảm biến DHT22 và module quản lý thời gian phần mềm hoạt động đúng logic theo thiết kế. Quá trình kiểm thử được thực hiện thủ công trên phần cứng thực tế, quan sát kết quả qua giao diện người dùng.

8.1.1. Kiểm thử driver DHT22

Driver DHT22 triển khai giao tiếp 1-Wire qua ba hàm chính: DHT22_Start() gửi tín hiệu khởi động, DHT22_Check_Response() nhận ACK từ cảm biến, và DHT22_Read_Byte() đọc từng byte dữ liệu. Hàm DHT22_ReadData() gọi tuần tự các hàm trên để đọc đủ 5 byte, kiểm tra checksum, rồi cập nhật biến Temperature và Humidity

Phương pháp kiểm thử: In giá trị Temperature và Humidity ra UART mỗi 2 giây, quan sát trực tiếp trên terminal. Đồng thời in trạng thái checksum để biết dữ liệu có hợp lệ hay không.

Bảng 8.1: Kiểm thử module DHT22

Test case	Mô tả	Cách kiểm tra	Kết quả kỳ vọng	Kết quả thực tế
1	Cảm biến kết nối và phản hồi bình thường.	Kiểm tra giá trị nhiệt độ trên giao diện người dùng.	Giá trị nhiệt độ và độ ẩm hợp lý, checksum OK.	PASS – Giá trị ổn định, phù hợp điều kiện phòng.
2	Rút cảm biến khỏi mạch.	Ngắt cảm biến.	Temperature không thay đổi (giữ giá trị cuối).	PASS – Biến không bị cập nhật do response = 0

Nhận xét: Driver hoạt động đúng luồng logic của giao thức DHT22. Cơ chế kiểm tra checksum trước khi cập nhật biến Temperature/Humidity đảm bảo hệ thống không sử dụng dữ liệu lỗi. Khi cảm biến bị ngắt, giá trị đo được giữ nguyên và không làm hỏng trạng thái điều khiển.

8.1.2. Kiểm thử module SoftTimer

Module SoftTimer triển khai bộ đếm thời gian phần mềm dựa trên ngắt SysTick 1 ms. Mỗi tick ISR gọi SoftTimer_Update() để đếm ngược, khi counter về 0 thì tăng biến event. Vòng lặp chính gọi SoftTimer_Dispatch() để xử lý sự kiện tồn đọng và gọi callback. Thiết kế này tách biệt hoàn toàn phần đếm thời gian (trong ISR) với phần xử lý logic (trong main loop).

Phương pháp kiểm thử: Đặt callback toggle LED, quan sát tần suất kích hoạt bằng mắt và đồng hồ bấm giây.

Bảng 8.2: Kiểm thử Software Timer

Test Case	Mô tả	Cách kiểm tra	Kết quả kỳ vọng	Kết quả thực tế
1	Timer kích hoạt callback đúng chu kỳ.	Toggle LED, bấm giờ 10 lần kích hoạt.	LED nhấp nháy đều đặn theo period đặt.	PASS – Nhịp đều, ước lượng đúng chu kỳ.
2	SoftTimer_Stop() dừng đúng.	Gọi Stop sau 5 s, quan sát LED.	LED tắt và không nhấp nháy nữa.	PASS – Callback không được gọi thêm.
3	SoftTimer_Start() lại sau Stop.	Gọi Start lại, quan sát LED.	LED nhấp nháy trở lại bình thường.	PASS – Hoạt động lại đúng chu kỳ.

Nhận xét: Module SoftTimer hoạt động đúng theo thiết kế. Đặc biệt, cơ chế tách Update/Dispatch giúp hệ thống không mất sự kiện ngay cả khi vòng lặp chính bị chiếm dụng bởi việc đọc cảm biến (~5 ms mỗi lần đọc DHT22). Biến event được giới hạn tối đa 255 để tránh tràn khi hệ thống bị trì hoãn lâu.

Kết luận: Tầng Sensor & Timer hoạt động đúng logic, sẵn sàng để tích hợp với tầng điều khiển đèn sưởi phía trên.

8.2. Kiểm thử tầng cơ cấu chấp hành

Đảm bảo cấu trúc dữ liệu điều khiển (ActuatorPrams_struct_t) từ tầng ứng dụng được chuyển đổi chính xác từ các đơn vị vật lý thực tế (% , RPM, độ C) sang chu kỳ nhiệm vụ (Duty Cycle) của xung PWM cấu hình cho phần cứng.

Đánh giá khả năng hoạt động và các kịch bản bảo vệ hệ thống khi xảy ra lỗi dữ liệu đầu vào.

Phương pháp kiểm thử: Sử dụng 02 đèn LED kết nối vào các chân PWM tương ứng để quan sát cường độ sáng

- LED 1 (Thay thế cho Quạt - FAN): Kết nối vào chân PA6
- LED 2 (Thay thế cho Máy sưởi - HEATER): Kết nối vào chân PA7

Bảng 8.3: Kiểm thử cơ cấu chấp hành

STT	Kịch bản kiểm thử	Thông số cấu hình đầu vào	Chu kỳ xung đo được (Duty Cycle)	Trạng thái vật lý của LED chỉ thị	Kết luận
1	Bật Quạt tối đa (ON)	FAN, ACT_MODE_ONOFF, value = 100.0f	100%	LED 1 (PA6) sáng liên tục với cường độ cực đại.	ĐẠT

2	Tắt Quạt (OFF)	FAN, ACT_MODE_ONOFF, value = 0.0f	0%	LED 1 (PA6) tắt hoàn toàn.	ĐẠT
3	Điều khiển Quạt tuyến tính	FAN, ACT_MODE_PWM, UNIT_PERCENT, value = 25.0f	25%	LED 1 (PA6) sáng mờ ổn định.	ĐẠT
4	Điều khiển Quạt tuyến tính	FAN, ACT_MODE_PWM, UNIT_PERCENT, value = 75.0f	75%	LED 1 (PA6) sáng mạnh rõ rệt.	ĐẠT
5	Quy đổi tốc độ Quạt (RPM)	FAN, ACT_MODE_PWM, UNIT_RPM, value = 1500.0f	50%	LED 1 (PA6) sáng ở mức trung bình.	ĐẠT
6	Quy đổi công suất bộ gia nhiệt	HEATER, ACT_MODE_PWM, UNIT_DEG_C, value = 40.0f	40%	LED 2 (PA7) sáng mờ vừa phải.	ĐẠT
7	Quy đổi công suất bộ gia nhiệt.	HEATER, ACT_MODE_PWM, UNIT_DEG_C, value = 90.0f	90%	LED 2 (PA7) sáng rất mạnh.	ĐẠT
8	Bắt lỗi phần mềm (Ngoại lệ)	HEATER, ACT_MODE_PWM, UNIT_DEG_C, value = 1000.0f	0% / 0%	Cả 2 LED ngay lập tức ngắt (Duty = 0) để bảo vệ hệ thống.	ĐẠT
9	Bắt lỗi phần mềm (Ngoại lệ)	mode = 5	0% / 0%	Cả 2 LED ngay lập tức ngắt (Duty = 0) để bảo vệ hệ thống.	Đạt

8.3. Kiểm thử tích hợp hệ thống và giao tiếp

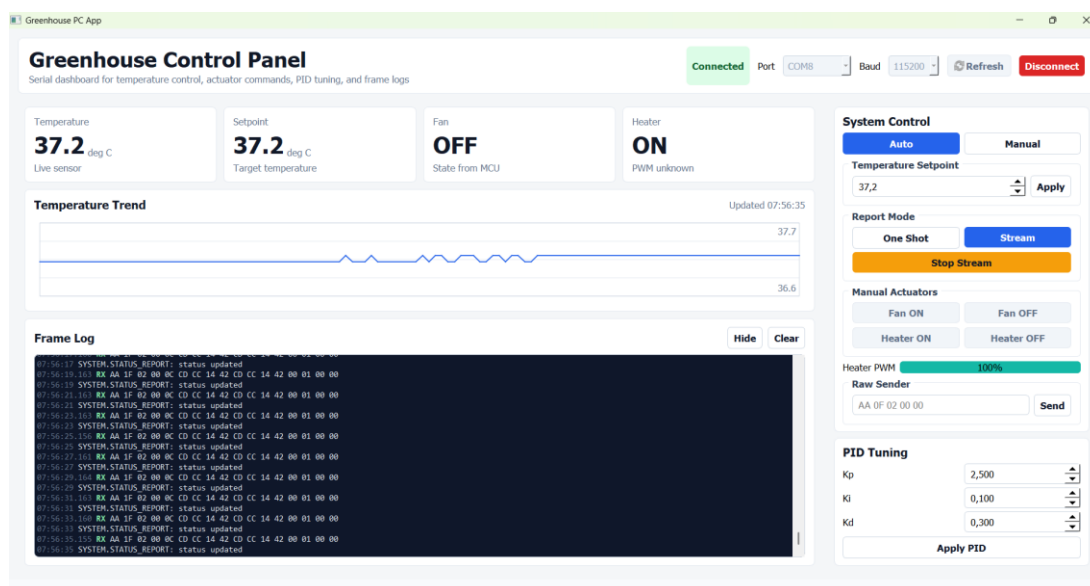
Mục tiêu: Đánh giá độ ổn định của kênh truyền UART và độ chính xác khi đóng gói/giải mã số thực giữa giao diện Python và Firmware C.

Bảng 8.4: Kiểm thử tích hợp hệ thống và giao tiếp

Hạng mục kiểm thử	Kịch bản / Thao tác thực hiện trên PC Tool	Phản hồi của hệ thống (MCU & PC)	Đánh giá
Giao tiếp cơ bản	Chọn đúng cổng COM, cấu hình Baudrate 115200 và nhấn nút Connect.	Hệ thống thiết lập liên kết thành công. PC báo trạng thái "Connected". Hàng đợi hai bên khởi tạo bình thường, không xuất hiện rác bộ đệm.	ĐẠT
Luồng dữ liệu liên tục	Chuyển Report Mode trên giao diện sang chế độ Stream.	MCU định kỳ 2 giây đóng gói gửi bản tin STATUS_REPORT (chứa nhiệt độ, setpoint, trạng thái). PC giải mã mượt mà và vẽ đồ thị nhiệt độ theo thời gian thực.	ĐẠT
Cập nhật điểm đặt (Setpoint)	Đổi giá trị Temperature Setpoint (từ 40°C sang 38.5°C) và nhấn Apply.	PC đóng gói giá trị Setpoint vào Payload và gửi frame SET_SETPOINT. MCU nhận, cập nhật vào RAM và gửi trả frame xác nhận RSP_SET_SETPOINT_OK. Đồ thị PC lập tức dời đường Setpoint.	ĐẠT

Cập nhật hệ số PID	Nhập bộ tham số mới cho Kp, Ki, Kd trên giao diện và nhấn Apply PID.	Lệnh SET_PID được gửi xuống. MCU nhận và thay đổi công suất băm xung của bộ gia nhiệt ngay lập tức dựa trên bộ hệ số mới.	ĐẠT
Điều khiển thủ công (Manual Mode)	Chuyển System Control sang Manual. Nhấn các nút bật/tắt Quạt (Fan) và Bộ gia nhiệt (Heater).	Các thiết bị chấp hành đóng ngắt tức thời theo lệnh từ PC. Log hiển thị Actuator updated.	ĐẠT
Ngoại lệ	Gửi lệnh không trong phạm vi xử lý của MCU.	Lệnh này thuộc kiểu RSP (MCU chỉ xử lý các lệnh CMD). MCU bỏ qua lệnh.	ĐẠT

Nhận xét: Ứng dụng trên máy tính giao tiếp ổn định với vi điều khiển. Khung truyền được thiết kế tối ưu giúp việc gửi/nhận số thập phân (float 32-bit của nhiệt độ, PID) hoàn toàn chính xác. Hệ thống hàng đợi xử lý chống nghẽn thành công.



Hình 8.1: Ảnh chụp phần Frame Log đang hiển thị chuỗi byte TX/RX liên tục

Video kiểm thử: [Video_test](#)

9. KẾT QUẢ ĐẠT ĐƯỢC

Sau quá trình thiết kế, lập trình và kiểm thử, hệ thống đã hoàn thành các mục tiêu đề ra về kiến trúc phần mềm, thuật toán điều khiển và phần mềm giám sát.

Về kiến trúc phần mềm, dự án đã xây dựng thành công Firmware theo kiến trúc phân tầng gồm Application, Platform, MCU Drivers và Common, giúp giảm sự phụ thuộc giữa các thành phần, thuận tiện cho việc bảo trì và mở rộng. Hệ thống giao tiếp UART ở tốc độ 115200 bps hoạt động ổn định nhờ kết hợp cơ chế hàng đợi và máy trạng thái hữu hạn.

Về điều khiển, hệ thống sử dụng cảm biến DHT22 kết hợp bộ điều khiển PID để duy trì nhiệt độ theo giá trị đặt. Kết quả kiểm thử cho thấy hệ thống đạt trạng thái ổn định với sai số xác lập khoảng $\pm 0,1^{\circ}\text{C}$ tại mức nhiệt độ đặt $37,2^{\circ}\text{C}$.

Về phần mềm giám sát, ứng dụng được phát triển bằng Python/PyQt5, hỗ trợ hiển thị dữ liệu thời gian thực, cấu hình tham số điều khiển và giao tiếp ổn định với vi điều khiển, đáp ứng tốt yêu cầu giám sát và kiểm thử của hệ thống.

10. TÓM TẮT GIÁ TRỊ VÀ BÀI HỌC KINH NGHIỆM

10.1. Tóm tắt giá trị và bài học kinh nghiệm

Dự án không chỉ xây dựng thành công hệ thống điều khiển nhiệt độ sử dụng vi điều khiển STM32 mà còn giúp nhóm tiếp cận quy trình phát triển phần mềm nhúng theo hướng module và phân tầng. Việc tổ chức mã nguồn thành các tầng Application, Platform, MCU Drivers và Common giúp hệ thống có cấu trúc rõ ràng, thuận tiện cho việc bảo trì và mở rộng.

Trong quá trình thực hiện, nhóm đã tích lũy được kinh nghiệm về lập trình ở mức thanh ghi, thiết kế giao tiếp UART, xây dựng bộ điều khiển PID, tổ chức phần mềm theo hướng không chặn và phối hợp giữa các module thông qua ngắt, hàng đợi và máy trạng thái. Đây là những kiến thức và kỹ năng quan trọng trong phát triển hệ thống nhúng thời gian thực.

10.2. Định hướng cải thiện và mở rộng

Trong tương lai, hệ thống có thể được nâng cấp bằng cách cải thiện thuật toán điều khiển PID nhằm rút ngắn thời gian xác lập và nâng cao chất lượng đáp ứng. Bên cạnh đó, giao thức truyền thông có thể được bổ sung cơ chế kiểm tra lỗi như Checksum hoặc CRC để tăng độ tin cậy khi truyền dữ liệu.

Ngoài ra, hệ thống có thể tích hợp các module truyền thông không dây như ESP8266 hoặc ESP32 để hỗ trợ giám sát và điều khiển từ xa thông qua nền tảng IoT. Việc sử dụng các cảm biến có độ chính xác cao hơn và thiết kế mạch phần cứng chuyên dụng cũng là những hướng phát triển phù hợp nhằm nâng cao hiệu năng và khả năng ứng dụng của hệ thống trong thực tế.

11. PHỤ LỤC

11.1. Bảng tra cứu Message ID

Hệ thống phân loại các lệnh giao tiếp thành từng nhóm (Group) để dễ quản lý và mở rộng.

Bảng 11.1: Bảng tra cứu Message ID

GID	MID	Tên bản tin	Chức năng
0x0 (SYSTEM)	0x1	SET_MODE	Chuyển đổi chế độ hoạt động (Auto/Manual).
	0x2	STATUS_REPORT	Yêu cầu hoặc báo cáo trạng thái toàn hệ thống.
	0x3	SET_SETPOINT	Cập nhật giá trị nhiệt độ cài đặt.
	0x4	SET_REPORT_MODE	Cấu hình chế độ gửi dữ liệu (One Shot / Stream).

0x1 (SENSOR)	0x1	SENSOR_REPORT	Báo cáo giá trị nhiệt độ hiện tại từ cảm biến.
0x2 (ACTUATOR)	0x1	MANUAL_ACTUATOR	Lệnh điều khiển thủ công (bật/tắt quạt, sưởi).
0x3 (PID)	0x1	SET_PID	Cập nhật các hệ số điều khiển (Kp, Ki, Kd).
0x4 (ERROR)	0x1 à 0x4	ACT_ERR_*	Báo cáo lỗi liên quan đến phần cứng Actuator.

11.2. Chi tiết cấu hình các chân của vi điều khiển

Bảng 11.2: Bảng cấu hình các chân của vi điều khiển

Tên Chân (Pin)	Cổng (Port)	Chức năng hệ thống	Bộ ngoại vi liên quan	Chế độ cấu hình (Mode / CNF)	Ghi chú
PA0	Port A	Đọc/Ghi dữ liệu Cảm biến	GPIO	Đảo liên tục: OUT50 + O_GP_PP <- > IN + I_PP	Kết nối cảm biến DHT22 (Giao tiếp 1 dây)
PA6	Port A	Điều khiển Quạt (Fan)	TIM3_CH1 (PWM)	Alternate Function Output Push-Pull	Tần số xung 2500 Hz, điều khiển tốc độ qua Duty Cycle
PA7	Port A	Điều khiển Máy sưởi (Heater)	TIM3_CH2 (PWM)	Alternate Function Output Push-Pull	Tần số xung 2500 Hz, điều khiển công suất qua Duty Cycle

PA9	Port A	Truyền dữ liệu lên PC (TX)	USART1_TX	Alternate Function Output Push-Pull	Tốc độ Baudrate cấu hình động, tốc độ tối đa chân 50MHz
PA10	Port A	Nhận dữ liệu từ PC (RX)	USART1_RX	Floating Input (Ngõ vào để nổi)	Nhận lệnh điều khiển, kích hoạt ngắt USART1_IRQn.

11.3. Kết quả hoạt động và mã nguồn dự án

Kết quả: [Video hoạt động](#)

Mã nguồn: [Source Code](#)

11.4. Danh mục từ viết tắt

Bảng 11.3: Bảng danh mục từ viết tắt

Viết tắt	Tiếng anh	Giải nghĩa tiếng việt
MCU	Microcontroller Unit	Vi điều khiển
GPIO	General Purpose Input/Output	Cổng vào/ra đa năng
UART	Universal Asynchronous Receiver/Transmitter	Chuẩn truyền thông nối tiếp không đồng bộ
PWM	Pulse Width Modulation	Điều chế độ rộng xung
PID	Proportional–Integral–Derivative	Bộ điều khiển tỉ lệ – tích phân – vi phân
HAL	Hardware Abstraction Layer	Lớp trừu tượng hóa phần cứng
API	Application Programming Interface	Giao diện lập trình ứng dụng
FSM	Finite State Machine	Máy trạng thái hữu hạn
PC	Personal Computer	Máy tính cá nhân

DHT22	Digital Humidity and Temperature Sensor	Cảm biến nhiệt độ và độ ẩm DHT22
USB	Universal Serial Bus	Chuẩn giao tiếp USB